Python Programming — Month 2

Project 3

Python Quiz Game

Complete Teaching Guide & Documentation

CloudExify Summer Internship 2026

Project	Python Quiz Game
Type	Interactive CLI Game
Language	Python 3.x
New Library	random (built-in — no install needed)
Difficulty	Beginner
Hours	7–8 hours
Submission	End of Week 2 of Month 2

1. Introduction

The Quiz Game is one of the most enjoyable Python beginner projects because you get to play what you build! It teaches you one very important new concept — the random module — while reinforcing everything from Month 1. By the end you will have a fully working game that you can share with friends.

The random module — what is it?

The random module is a built-in Python library (no installation needed!) that lets you introduce randomness into your programs. You will use it to shuffle questions so they appear in different order every time the game is played.


```
import random

# Pick a random number between 1 and 10
num = random.randint(1, 10)
print(num)

# Pick a random item from a list
fruits = ["apple", "banana", "mango"]
picked = random.choice(fruits)
print(picked)

# Shuffle a list (changes order randomly)
questions = [1, 2, 3, 4, 5]
random.shuffle(questions)
print(questions)  # e.g. [3, 1, 5, 2, 4]
```

2. Concepts You Will Use

Concept	Used For
import random	Shuffling questions randomly
List of dictionaries	Storing question bank
For loops	Going through each question
If/elif/else	Checking answers, assigning grade
Functions	Organizing each feature
File handling	Saving and loading high score
String .upper()	Case-insensitive answer check
Counter variables	Tracking score
While loop	Play again loop

3. Project Structure

```
quiz_game/
    ■■■ quiz_game.py       # Main game file
    ■■■ highscore.txt      # Saved high score
    ■■■ README.md          # Project description
```

4. Step-by-Step Guide

Step 1 — Create the question bank (Part A)

Each question is a dictionary with question text, 4 options, and the correct answer letter. Save as quiz_game.py:

```
# quiz_game.py  — CloudExify Python Internship Month 2 P3
import random

# Each question: text + 4 options + correct answer
QUESTIONS = [
    {
        "question": "What is the output of: print(2 ** 3)?",
        "options": {"A":"6", "B":"8", "C":"9", "D":"23"},
        "answer": "B"
    },
    {
        "question": "Which keyword defines a function?",
        "options": {"A":"function","B":"define","C":"def","D":"func"},
        "answer": "C"
    },
    {
        "question": "What data type is: x = [1, 2, 3]?",
        "options": {"A":"tuple","B":"dict","C":"string","D":"list"},
        "answer": "D"
    },
    {
        "question": "How do you get user input?",
        "options": {"A":"get()","B":"input()","C":"read()","D":"scan()"},
        "answer": "B"
    },
    {
        "question": "What does len([1, 2, 3, 4]) return?",
        "options": {"A":"3","B":"5","C":"4","D":"0"},
        "answer": "C"
    },
]
# ADD 10+ MORE QUESTIONS using the same format!
# Topics: if/else, loops, strings, files, dicts...
```

Step 1 continued — More questions (Part B)


```
# Add these to your QUESTIONS list:
    {
        "question": "Which loop runs while a condition is True?",
        "options": {"A":"for","B":"while","C":"if","D":"do"},
        "answer": "B"
    },
    {
        "question": "How do you create a comment in Python?",
        "options": {"A":"//","B":"/**/","C":"#","D":"--"},
        "answer": "C"
    },
    {
        "question": "What does print(type(3.14)) output?",
        "options": {"A":"int","B":"float","C":"str","D":"num"},
        "answer": "B"
    },
    {
        "question": "How do you open a file for reading?",
        "options": {"A":"open('f','w')","B":"open('f','a')",
                    "C":"open('f','r')","D":"open('f','x')"},
        "answer": "C"
    },
    {
        "question": "What is output of: print('Hello'[0])?",
        "options": {"A":"Hello","B":"H","C":"e","D":"0"},
        "answer": "B"
    },
]
# ADD 5 MORE QUESTIONS YOURSELF using same format!
# Hint: Ask about if statements, range(), lists, etc.
```

Step 2 — Display and ask question


```
def ask_question(question_data, q_number, total):
    print(f"\nQuestion {q_number} of {total}")
    print("-" * 40)
    print(question_data["question"])
    print()

    # Show all 4 options
    for letter, option in question_data["options"].items():
        print(f"   {letter}) {option}")

    print()

    # Get answer with validation
    while True:
        answer = input("Your answer (A/B/C/D): ").strip().upper()
        if answer in ["A", "B", "C", "D"]:
            break
        print("Please enter A, B, C, or D only!")

    # Check if correct
    correct = question_data["answer"]
    if answer == correct:
        print("CORRECT! Well done!")
        return True
    else:
        correct_text = question_data["options"][correct]
        print(f"Wrong! Correct answer was {correct}) {correct_text}")
        return False
```


Step 3 — Calculate grade

```
def get_grade(score, total):
    percentage = (score / total) * 100

    if percentage >= 90:
        return "A", "Excellent! Outstanding performance!"
    elif percentage >= 80:
        return "B", "Great job! Very good performance!"
    elif percentage >= 70:
        return "C", "Good. You passed with decent marks."
    elif percentage >= 60:
        return "D", "You passed but needs improvement."
    else:
        return "F", "You did not pass. Keep practicing!"

def show_results(score, total):
    percentage = (score / total) * 100
    grade, message = get_grade(score, total)

    print("\n" + "=" * 40)
    print("      QUIZ COMPLETED!")
    print("=" * 40)
    print(f"  Score      : {score} / {total}")
    print(f"  Percentage : {percentage:.1f}%")
    print(f"  Grade      : {grade}")
    print(f"  Result     : {message}")
    print("=" * 40)
```

Step 4 — High score system

```
def load_high_score():
    try:
        with open("highscore.txt", "r") as f:
            return int(f.read().strip())
    except (FileNotFoundError, ValueError):
        return 0

def save_high_score(score):
    current_high = load_high_score()
    if score > current_high:
        with open("highscore.txt", "w") as f:
            f.write(str(score))
        print(f"NEW HIGH SCORE: {score}!")
        return True
    return False
```

Step 5 — Main game function


```
def play_quiz():
    # Shuffle questions for random order
    questions = QUESTIONS.copy()
    random.shuffle(questions)

    # Use only 10 questions per game
    game_questions = questions[:10]

    score = 0
    total = len(game_questions)
    high_score = load_high_score()

    print("=" * 40)
    print("  CLOUDEXIFY PYTHON QUIZ GAME")
    print("=" * 40)
    print(f"  Questions: {total}")
    print(f"  High Score: {high_score}")
    print("  Answer with A, B, C, or D")
    print("=" * 40)
    input("  Press Enter to start...")

    for i, question in enumerate(game_questions, 1):
        is_correct = ask_question(question, i, total)
        if is_correct:
            score += 1

    show_results(score, total)
    save_high_score(score)

def main():
    while True:
        play_quiz()
        print()
        again = input("Play again? (yes/no): ").strip().lower()
        if again not in ["yes", "y"]:
            print("Thanks for playing! Goodbye!")
            break

if __name__ == "__main__":
    main()
```

5. Testing Checklist

Test Case	Expected Result	Done
Run game — 10 questions shown	Questions in random order	■
Run again — different order	Questions shuffled differently	■
Enter wrong letter (e.g. E)	Error message, ask again	■
Answer all correctly	Score 10/10, Grade A	■
Answer all wrong	Score 0/10, Grade F	■

Beat previous high score	New high score message	■
Play again after finishing	New game starts fresh	■
Choose not to play again	Goodbye message, exits	■

6. Bonus Challenges

Bonus	Difficulty
Add timer — 30 seconds per question	Easy
Add difficulty levels (Easy/Hard questions)	Medium
Show correct answers review at end	Easy
Add player name and save score leaderboard	Medium
Add 5 more questions of your own	Easy

■ HOW TO SUBMIT: 1. Push code to GitHub Repository name: cloudexify-python-p3-yourname Must include: main source file + README.md 2. README.md must include: Your name · Registration number · Features list · How to run 3. Include at least 2 screenshots of your running program 4. WhatsApp your PM: [CX-2026-XXXX] Project 3 Done — [GitHub Link]